

Word Ladder

Company: Amazon

Round: Technical Interview

Year: 2025

Difficulty: Hard

Topic: Graph, Breadth-First Search (BFS), Shortest Path

Source: LeetCode 127 - Word Ladder

Problem Description

A transformation sequence from word **beginWord** to word **endWord** using a dictionary **wordList** is a sequence of words **beginWord** → **s1** → **s2** → ... → **sk** such that:

- Every adjacent pair of words differs by a single letter.
- Every s_i for $1 \leq i \leq k$ is in **wordList**. Note that **beginWord** does not need to be in **wordList**.
- $s_k == \text{endWord}$.

Given two words, **beginWord** and **endWord**, and a dictionary **wordList**, return the number of words in the shortest transformation sequence from **beginWord** to **endWord**, or 0 if no such sequence exists.

Input Format

The first line contains two strings — **beginWord** and **endWord**.

The second line contains an integer **N** — the number of words in the dictionary.

The third line contains **N** space-separated strings denoting **wordList**.

Output Format

Print a single integer — the number of words in the shortest valid transformation sequence. Print **0** if no valid transformation sequence exists.

Constraints

- $1 \leq \text{beginWord.length} \leq 10$
- $\text{endWord.length} == \text{beginWord.length}$
- $1 \leq \text{wordList.length} \leq 5000$
- $\text{wordList}[i].\text{length} == \text{beginWord.length}$
- **beginWord**, **endWord**, and **wordList[i]** consist of lowercase English letters.
- **beginWord** != **endWord**
- All the words in **wordList** are unique.

Sample Input / Output

Example 1

Input:

beginWord = "hit"

endWord = "cog"

wordList = ["hot","dot","dog","lot","log","cog"]

Output: 5

Explanation: One shortest transformation sequence is "hit" → "hot" → "dot" → "dog" → "cog", which is 5 words long.

Example 2

Input:

beginWord = "hit"

endWord = "cog"

wordList = ["hot", "dot", "dog", "lot", "log"]

Output: 0

Explanation: The endWord "cog" is not in wordList, therefore there is no valid transformation sequence.

Approach to Solve This Problem

The key idea is to model each word as a node in a graph. Two words are connected when they differ by exactly one character. The required answer is the shortest path from beginWord to endWord, so **Breadth-First Search (BFS)** is the natural approach.

1. Check whether endWord exists

If endWord is not present in wordList, immediately return 0 because every intermediate and final transformed word must belong to the dictionary.

2. Initialize BFS

Insert beginWord into a queue with transformation length 1. Maintain a visited set so that the same word is not processed repeatedly.

3. Generate one-letter transformations

For each word removed from the queue, change each character position to every letter from a to z. Whenever the generated word exists in the dictionary and has not been visited, add it to the queue with distance + 1.

4. Stop at endWord

As soon as endWord is reached, return its BFS level. Because BFS explores states level by level, the first time endWord is found gives the shortest transformation sequence.

5. No valid transformation

If the queue becomes empty before reaching endWord, return 0.

Java Solution

```
import java.util.*;

class Solution {
    public int ladderLength(String beginWord, String endWord,
        List<String> wordList) {

        Set<String> dict = new HashSet<>(wordList);

        if (!dict.contains(endWord)) {
            return 0;
        }

        Queue<String> queue = new LinkedList<>();
        Set<String> visited = new HashSet<>();

        queue.offer(beginWord);
        visited.add(beginWord);
```

```
int level = 1;

while (!queue.isEmpty()) {
    int size = queue.size();

    for (int s = 0; s < size; s++) {
        String word = queue.poll();

        if (word.equals(endWord)) {
            return level;
        }

        char[] chars = word.toCharArray();

        for (int i = 0; i < chars.length; i++) {
            char original = chars[i];

            for (char c = 'a'; c <= 'z'; c++) {
                if (c == original) continue;

                chars[i] = c;
                String next = new String(chars);

                if (dict.contains(next) && !visited.contains(next)) {
                    visited.add(next);
                    queue.offer(next);
                }

                chars[i] = original;
            }

            level++;
        }

        return 0;
    }
}
```

Complexity Analysis

Time Complexity: $O(N \times L \times 26)$, where N is the number of words and L is the word length. Each visited word can generate up to $26 \times L$ candidate transformations.

Space Complexity: $O(N)$, for the dictionary, visited set, and BFS queue.

Interview Notes

- BFS is preferred because the problem asks for the **shortest** transformation sequence in an unweighted graph.
- Checking endWord before BFS avoids unnecessary work when the target is absent.
- The visited set is important to prevent cycles and repeated processing.
- A common optimization is to use wildcard patterns such as **h*t**, ***it**, etc., to build adjacency efficiently.
- Another strong interview approach is **bidirectional BFS**, starting from both beginWord and endWord, which can significantly reduce the search space.